CONVERGENCE DE LA CROSS-ENTROPY VERS L'ENTROPIE DE SHANNON

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

20 octobre 2026

Table des matières

I Cours Magistral **2**

1 Contexte Historique et Physique de l'Information **2**

2 Structures Mathématiques Fondamentales **2**
2.1 Définitions, Théorèmes et Exemples Concrets Immédiats 2

3 Démonstration Exhaustive de la Convergence Empirique **4**

4 Applications à l'Intelligence Artificielle et à la Physique **5**

II Exercices d'Application et de Concours **6**

III Travaux Pratiques et Simulations Algorithmiques **13**

Première partie

Cours Magistral

1 Contexte Historique et Physique de l'Information

L'étude de l'information trouve ses racines dans la physique statistique, notamment via les travaux de Ludwig Boltzmann. C'est Claude Shannon, en 1948, qui formalise mathématiquement la notion d'incertitude dans un processus aléatoire. En intelligence artificielle moderne, l'entraînement des modèles de langage massifs (LLM) repose sur la minimisation d'une fonction de perte appelée entropie croisée (ou Cross-Entropy).

Ce processus d'optimisation vise à rapprocher la distribution de probabilité prédite par le modèle de la distribution réelle du langage humain. Nous démontrerons que cette perte est bornée inférieurement par l'entropie intrinsèque du langage : l'incertitude irréductible inhérente à la communication humaine.

2 Structures Mathématiques Fondamentales

Soit $(\mathcal{X}, \mathcal{B}(\mathcal{X}), \lambda)$ un espace mesuré, où $\mathcal{X}$ représente l'espace de nos données (par exemple, le vocabulaire d'une langue). Soit P la distribution réelle et inconnue des données, absolument continue par rapport à λ, de densité p. Soit Q_θ la distribution de probabilité paramétrée par $\theta \in \Theta$, prédite par notre modèle, de densité q_θ.

2.1 Définitions, Théorèmes et Exemples Concrets Immédiats

¿ **Définition 1 (Entropie de Shannon) :** ¿ L'entropie de la source de données P est définie par : ¿

$$H(P) = -\int_{\mathcal{X}} p(x) \ln p(x) d\lambda(x) = \mathbb{E}_{X\sim P}[-\ln p(X)]$$

¿ Cette grandeur mesure la quantité moyenne d'information (ou d'incertitude) intrinsèque aux données.

Exemple 1 : Lancer de pièce biaisée Considérons une variable aléatoire binaire $X \in \{0, 1\}$ suivant une loi de Bernoulli de paramètre $p = 0,2$. La distribution est $P(X = 1) = 0,2$ et $P(X = 0) = 0,8$. L'entropie de Shannon de cette source est :

$$H(P) = -(0,2\ln(0,2) + 0,8\ln(0,8))$$

Calculons pas-à-pas :

$$0,2\ln(0,2) \approx 0,2 \times (-1,609) \approx -0,3218$$

$$0,8\ln(0,8) \approx 0,8 \times (-0,223) \approx -0,1784$$

$$H(P) \approx 0,3218 + 0,1784 = 0,5002 \text{ nats.}$$

¿ **Définition 2 (Entropie Croisée ou Cross-Entropy) :** ¿ La cross-entropy entre la vraie distribution P et la distribution approchée Q_θ est définie par : ¿

$$H(P, Q_\theta) = -\int_{\mathcal{X}} p(x) \ln q_\theta(x) d\lambda(x) = \mathbb{E}_{X\sim P}[-\ln q_\theta(X)]$$

Exemple 2 : Entropie croisée pour un modèle imparfait Reprenons la pièce biaisée P (Bernoulli $p = 0,2$). Supposons qu'un modèle naïf Q_θ estime que la pièce est équilibrée : $q(0) = 0,5$ et $q(1) = 0,5$. La cross-entropy est :

$$H(P, Q_\theta) = -(P(X = 1) \ln q(1) + P(X = 0) \ln q(0))$$

$$H(P, Q_\theta) = -(0,2 \ln(0,5) + 0,8 \ln(0,5)) = -\ln(0,5) = \ln(2) \approx 0,693 \text{ nats.}$$

On remarque que $H(P, Q_\theta) > H(P)$. Le modèle équilibré est "surpris" par les résultats réels plus souvent qu'un modèle parfait ne le serait.

Exemple 3 : Entropie d'un système déterministe Considérons une source P qui émet toujours le même symbole $X = A$ avec certitude : $P(X = A) = 1$ et $P(X \neq A) = 0$. L'entropie de cette source est :

$$H(P) = -(1 \ln(1)) = 0$$

L'incertitude est nulle. On ne peut extraire aucune information d'un tel système.

Exemple 4 : Cross-Entropy infinie Soit un modèle Q_θ qui affirme avec certitude qu'une pièce tombera sur "Pile" : $q(Pile) = 1, q(Face) = 0$. Si la vraie distribution P est une pièce équilibrée $P(Pile) = 0,5, P(Face) = 0,5$. La cross-entropy est :

$$H(P, Q_\theta) = -(0,5 \ln(1) + 0,5 \ln(0))$$

Or $\ln(0) \to -\infty$. La fonction de perte devient infinie. Le modèle est infiniment surpris et donc infiniment pénalisé pour avoir attribué une probabilité nulle à un événement réel.

Exemple 5 : Divergence KL et compression Pour compresser des données issues d'une source P (Entropie $H(P) = 2$ bits), on utilise un encodeur optimisé pour une loi Q. La cross-entropy est mesurée à $H(P, Q) = 2,5$ bits. La perte d'efficacité par symbole est exactement la divergence KL :

$$D_{KL}(P\|Q) = 2,5 - 2 = 0,5 \text{ bits/symbole.}$$

C'est le coût direct de modéliser P par Q.

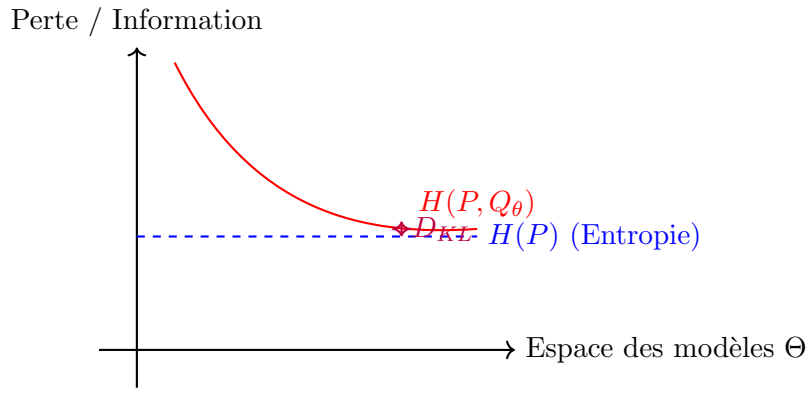
Exemple 6 : Optimisation d'un modèle de langage Un LLM naïf prédit chaque lettre de l'alphabet latin (26 lettres) de manière uniforme : $q = 1/26$. La cross-entropy avec un texte français P est :

$$H(P, Q) = -\sum_x p(x) \ln(1/26) = \ln(26) \approx 3,25 \text{ nats.}$$

Après entraînement sur un corpus, le modèle Q^* apprend la vraie distribution P. L'entropie de Shannon du français est estimée à environ $H(P) \approx 2,1$ nats. La perte du modèle va chuter de 3,25 à 2,1, atteignant le plancher théorique d'incertitude.

Exemple 7 : Calcul sur une distribution continue (Entropie différentielle) Soit une variable aléatoire X suivant une loi uniforme sur $[0, 2]$. La densité est $p(x) = 0,5$ sur ce segment. Son entropie différentielle est :

$$H(X) = -\int_0^2 0,5 \ln(0,5) dx = -\ln(0,5) \int_0^2 0,5 dx = -\ln(0,5) \times 1 = \ln(2) \approx 0,69 \text{ nats.}$$



¿ **Théorème 1 (Borne inférieure de l'entropie croisée) :** ¿ Pour toute distribution Q_θ, l'entropie croisée est majorée inférieurement par l'entropie de Shannon : ¿

$$H(P, Q_\theta) \geq H(P)$$

¿ De plus, $H(P, Q_\theta) = H(P)$ si et seulement si $Q_\theta = P$ presque partout.

Démonstration : On introduit la divergence de Kullback-Leibler $D_{KL}(P\|Q_\theta)$:

$$D_{KL}(P\|Q_\theta) = \int_{\mathcal{X}} p(x) \ln\left(\frac{p(x)}{q_\theta(x)}\right) d\lambda(x)$$

Par l'inégalité de Jensen appliquée à la fonction convexe $-\ln(t)$:

$$D_{KL}(P\|Q_\theta) = -\int_{\mathcal{X}} p(x) \ln\left(\frac{q_\theta(x)}{p(x)}\right) d\lambda(x) \geq -\ln\left(\int_{\mathcal{X}} p(x)\frac{q_\theta(x)}{p(x)} d\lambda(x)\right) = -\ln\left(\int_{\mathcal{X}} q_\theta(x) d\lambda(x)\right) = -\ln(1) =$$

En développant $D_{KL}(P\|Q_\theta)$:

$$D_{KL}(P\|Q_\theta) = \int_{\mathcal{X}} p(x) \ln p(x) d\lambda(x) - \int_{\mathcal{X}} p(x) \ln q_\theta(x) d\lambda(x) = -H(P) + H(P, Q_\theta)$$

Par conséquent :

$$H(P, Q_\theta) = H(P) + D_{KL}(P\|Q_\theta)$$

Comme $D_{KL}(P\|Q_\theta) \geq 0$, nous obtenons $H(P, Q_\theta) \geq H(P)$. La stricte convexité de $-\ln$ assure l'égalité ssi $P = Q_\theta$ p.p.

3 Démonstration Exhaustive de la Convergence Empirique

Dans un cadre d'apprentissage automatique, nous ne connaissons pas P, nous ne disposons que d'un ensemble de données d'entraînement $\{x_1, \ldots, x_n\}$ tiré de manière indépendante et identiquement distribuée (i.i.d.) selon P. L'entraînement du modèle consiste à minimiser la **perte empirique** (Negative Log-Likelihood) :

$$\hat{H}_n(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \ln q_\theta(x_i)$$

¿ **Théorème 2 (Convergence presque sûre de la perte empirique) :** ¿ Soit θ fixé. Sous la condition d'intégrabilité $\mathbb{E}_{X\sim P}[|\ln q_\theta(X)|] < \infty$, nous avons la convergence presque sûre : ¿

$$\hat{H}_n(\theta) \xrightarrow{p.s.} H(P, Q_\theta)$$

¿ lorsque $n \to \infty$.

Démonstration pas-à-pas : 1. Considérons les variables aléatoires $Y_i = -\ln q_\theta(x_i)$ pour $i \in \{1, \ldots, n\}$. 2. Puisque les variables x_i sont i.i.d. selon la loi P, les compositions $Y_i = -\ln q_\theta(x_i)$ sont également des variables aléatoires indépendantes et identiquement distribuées. 3. Calculons l'espérance mathématique de la variable Y_1 :

$$\mathbb{E}[Y_1] = \mathbb{E}_{X \sim P}[-\ln q_\theta(X)]$$

4. Par définition de l'entropie croisée, nous avons exactement :

$$\mathbb{E}[Y_1] = H(P, Q_\theta)$$

5. L'hypothèse $\mathbb{E}_{X \sim P}[|\ln q_\theta(X)|] < \infty$ implique que $\mathbb{E}[|Y_1|] < \infty$. 6. Nous pouvons alors invoquer la Loi Forte des Grands Nombres de Kolmogorov (LFGN, Jalon 92) sur la suite $(Y_i)_{i \geq 1}$:

$$\frac{1}{n} \sum_{i=1}^{n} Y_i \xrightarrow{p.s.} \mathbb{E}[Y_1]$$

7. En remplaçant par les définitions :

$$-\frac{1}{n} \sum_{i=1}^{n} \ln q_\theta(x_i) \xrightarrow{p.s.} H(P, Q_\theta)$$

Soit $\hat{H}_n(\theta) \xrightarrow{p.s.} H(P, Q_\theta)$.

Cette démonstration rigoureuse montre que, sur de grands jeux de données, la minimisation de la perte empirique équivaut à minimiser l'entropie croisée théorique, rapprochant ainsi le modèle de l'entropie asymptotique de Shannon.

4 Applications à l'Intelligence Artificielle et à la Physique

- **Limite Fondamentale de l'Apprentissage :** Si la vraie distribution des données P contient du bruit stochastique, $H(P) > 0$. Un modèle de langage massif ne pourra jamais atteindre une perte nulle. L'entraînement atteindra un plateau dicté par l'incertitude irréductible du langage. - **Théorie du Codage :** D'après le théorème de codage source de Shannon, $H(P, Q_\theta)$ représente la longueur moyenne du code (en nats ou bits) pour compresser les données en utilisant le modèle Q_θ, tandis que $H(P)$ est la limite théorique de compression. Optimiser un modèle de langage revient strictement à optimiser un compresseur de données.

Deuxième partie

Exercices d'Application et de Concours

Exercice 1 : Calcul élémentaire d'entropie

★⋆⋆⋆⋆

Soit une source émettant les symboles $\{A, B, C, D\}$ avec les probabilités $p(A) = 0,5$, $p(B) = 0,25$, $p(C) = 0,125$, $p(D) = 0,125$.

1. Calculer l'entropie de Shannon $H(P)$ de cette source en base 2 (en bits). **2.** Vérifier que cette valeur respecte la borne supérieure pour un alphabet de taille 4.

Correction :

1. Par définition, l'entropie de Shannon discrète en base 2 est :

$$H(P) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

Calculons chaque terme : Pour A : $-0,5 \log_2(0,5) = -0,5 \times (-1) = 0,5$ Pour B : $-0,25 \log_2(0,25) = -0,25 \times (-2) = 0,5$ Pour C : $-0,125 \log_2(0,125) = -0,125 \times (-3) = 0,375$ Pour D : $-0,125 \log_2(0,125) = -0,125 \times (-3) = 0,375$ En sommant :

$$H(P) = 0,5 + 0,5 + 0,375 + 0,375 = 1,75 \text{ bits.}$$

2. La borne supérieure de l'entropie pour un alphabet de taille $N = 4$ est atteinte pour la distribution uniforme, valant $\log_2(4) = 2$ bits. Nous avons bien $H(P) = 1,75 \leq 2$.

Exercice 2 : Cross-Entropy avec un modèle uniforme

★★⋆⋆⋆

On reprend la source P de l'exercice précédent. Un modèle d'apprentissage initialisé aléatoirement prédit une distribution uniforme Q sur les 4 symboles.

1. Calculer la cross-entropy $H(P, Q)$. **2.** En déduire la divergence de Kullback-Leibler $D_{KL}(P \| Q)$.

Correction :

1. Le modèle uniforme attribue $q(x) = \frac{1}{4} = 0,25$ pour tout x. La cross-entropy en base 2 est :

$$H(P, Q) = -\sum_{x} p(x) \log_2 q(x)$$

Puisque $q(x)$ est constant, nous avons :

$$H(P, Q) = -\sum_{x} p(x) \log_2(0,25) = -\log_2(0,25) \sum_{x} p(x)$$

Puisque $\sum_x p(x) = 1$, la cross-entropy est :

$$H(P, Q) = -\log_2(2^{-2}) = 2 \text{ bits.}$$

2. Nous savons par le théorème fondamental de ce cours que :

$$H(P, Q) = H(P) + D_{KL}(P \| Q)$$

D'après l'exercice précédent, $H(P) = 1,75$ bits. Par conséquent :

$$D_{KL}(P \| Q) = H(P, Q) - H(P) = 2 - 1,75 = 0,25 \text{ bits.}$$

Ce résultat quantifie exactement l'inefficacité du modèle uniforme par rapport à la vraie distribution.

Exercice 3 : Entropie d'une loi de Poisson

★★⋆⋆⋆

Soit X une variable aléatoire suivant une loi de Poisson de paramètre $\lambda > 0$, $X \sim \mathcal{P}(\lambda)$.

1. Exprimer l'entropie de Shannon $H(X)$ sous forme d'une série. **2.** Montrer que si λ est très grand, une approximation par la formule de Stirling pourrait lier cette entropie à l'entropie d'une loi normale discrétisée.

Correction :

1. La fonction de masse de probabilité de la loi de Poisson est $P(X = k) = e^{-\lambda}\frac{\lambda^k}{k!}$ pour $k \in \mathbb{N}$. L'entropie de Shannon (en nats) est :

$$H(X) = -\sum_{k=0}^{\infty} P(X = k)\ln(P(X = k))$$

Développons le logarithme :

$$\ln(P(X = k)) = \ln(e^{-\lambda}\frac{\lambda^k}{k!}) = -\lambda + k\ln\lambda - \ln(k!)$$

En substituant dans l'expression de l'entropie :

$$H(X) = -\sum_{k=0}^{\infty} P(X = k)(-\lambda + k\ln\lambda - \ln(k!))$$

Par la linéarité de l'espérance, on obtient :

$$H(X) = \lambda - (\mathbb{E}[X]\ln\lambda) + \mathbb{E}[\ln(X!)]$$

Puisque pour une loi de Poisson, $\mathbb{E}[X] = \lambda$, cela se simplifie en :

$$H(X) = \lambda(1 - \ln\lambda) + e^{-\lambda}\sum_{k=0}^{\infty}\frac{\lambda^k \ln(k!)}{k!}$$

2. Pour $\lambda \to \infty$, l'approximation de Stirling donne $\ln(k!) \approx k\ln k - k$. De plus, la loi de Poisson converge vers une loi normale $\mathcal{N}(\lambda, \lambda)$. L'entropie tend asymptotiquement vers celle d'une gaussienne, soit $H(X) \approx \frac{1}{2}\ln(2\pi e\lambda)$.

Exercice 4 : Minimisation empirique (Maximum de Vraisemblance)

★★★⋆⋆

Soit un échantillon $x_1, \ldots, x_n$ indépendant et identiquement distribué, tiré d'une loi de Bernoulli de paramètre p inconnu. On utilise un modèle paramétrique avec une distribution Bernoulli de paramètre $\theta \in]0, 1[$.

1. Écrire la fonction de perte Cross-Entropy empirique $\hat{H}_n(\theta)$. **2.** Montrer que minimiser $\hat{H}_n(\theta)$ revient à effectuer une estimation par maximum de vraisemblance, et trouver la valeur optimale $\hat{\theta}$.

Correction :

1. La distribution prédite pour un élément $x_i \in \{0, 1\}$ est $q_\theta(x_i) = \theta^{x_i}(1-\theta)^{1-x_i}$. La fonction de perte Cross-Entropy empirique (qui est la Negative Log-Likelihood moyenne) est :

$$\hat{H}_n(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\ln(q_\theta(x_i))$$

$$\hat{H}_n(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[x_i \ln\theta + (1-x_i)\ln(1-\theta)]$$

2. La minimisation de cette quantité est strictement équivalente à la maximisation de l'expression opposée :

$$\frac{1}{n}\sum_{i=1}^{n}\ln(q_\theta(x_i)) = \frac{1}{n}\ln\left(\prod_{i=1}^{n} q_\theta(x_i)\right)$$

Ceci est exactement la log-vraisemblance moyenne (Maximum de Vraisemblance). Pour trouver l'optimum $\hat{\theta}$, dérivons $\hat{H}_n(\theta)$ par rapport à θ :

$$\frac{\partial}{\partial\theta}\hat{H}_n(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[\frac{x_i}{\theta} - \frac{1-x_i}{1-\theta}\right]$$

On annule la dérivée :

$$\frac{1}{n}\sum_{i=1}^{n}\frac{x_i}{\theta} = \frac{1}{n}\sum_{i=1}^{n}\frac{1-x_i}{1-\theta}$$

Soit $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ la moyenne empirique. On obtient :

$$\frac{\bar{x}}{\theta} = \frac{1-\bar{x}}{1-\theta}$$

$$\bar{x}(1-\theta) = \theta(1-\bar{x}) \implies \bar{x} - \bar{x}\theta = \theta - \theta\bar{x} \implies \theta = \bar{x}$$

La valeur optimale est la moyenne empirique $\hat{\theta} = \bar{x}$.

Exercice 5 : Perplexité et Entropie

★★★⋆⋆

En traitement du langage naturel (NLP), la qualité d'un modèle de langage est souvent mesurée par sa perplexité, définie comme $PP = \exp(H(P,Q))$.

Soit une langue imaginaire constituée de seulement $N = 10^4$ mots possibles. On suppose que la distribution réelle P est uniforme sur l'ensemble de ces mots. Un modèle imparfait Q modélise cette distribution comme uniforme sur un sous-ensemble strict de $K = 10^3$ mots (et 0 pour les autres).

1. Calculer la perplexité du modèle parfait ($Q = P$). **2.** Pourquoi la perplexité du modèle imparfait Q est-elle infinie (et l'entropie croisée infinie) ?

Correction :

1. Pour le modèle parfait, $Q = P$. La distribution est uniforme sur N mots, donc $p(x) = \frac{1}{N}$. L'entropie croisée est :

$$H(P,P) = H(P) = -\sum_{x} p(x)\ln p(x) = -\sum_{x=1}^{N}\frac{1}{N}\ln\left(\frac{1}{N}\right) = \ln(N)$$

La perplexité est donc :

$$PP = \exp(H(P)) = \exp(\ln(N)) = N = 10^4$$

La perplexité d'une distribution uniforme est exactement le nombre de choix possibles.

2. Pour le modèle imparfait Q, il existe au moins un mot x^* tel que $p(x^*) = \frac{1}{N} > 0$ mais $q(x^*) = 0$ (car le support de Q est plus petit que celui de P). L'entropie croisée contient le terme :

$$-p(x^*)\ln(q(x^*)) = -\frac{1}{N}\ln(0)$$

Or $\lim_{z \to 0^+} \ln(z) = -\infty$, donc le terme tend vers $+\infty$. L'entropie croisée est infinie. Par conséquent, la perplexité l'est aussi. Cela illustre un principe de base en IA : un modèle ne doit jamais assigner une probabilité strictement nulle à un événement possible, sinon il encourt une pénalité infinie s'il est surpris par la réalité.

Exercice 6 : Lien avec la classification Multi-Classe

★★★⋆⋆

Dans le cas d'une classification à K classes, l'étiquette réelle d'un échantillon est un vecteur *one-hot* $y \in \{0,1\}^K$ où exactement une composante vaut 1 (disons $y_c = 1$ pour la classe cible c). La prédiction du modèle est un vecteur de probabilités $\hat{y} = (q_1, \ldots, q_K)$ tel que $\sum q_k = 1$.

1. Réécrire la formule de la Cross-Entropy discrète en fonction de y et $\hat{y}$. **2.** Simplifier cette expression en utilisant la propriété du vecteur *one-hot*.

Correction :

1. La vraie distribution pour cet échantillon est empirique et déterministe, donnée par les probabilités $p(k) = y_k$. Le modèle prédit $q(k) = q_k$. La formule de l'entropie croisée pour ce point de donnée est :

$$H(y, \hat{y}) = -\sum_{k=1}^{K} p(k) \ln(q(k)) = -\sum_{k=1}^{K} y_k \ln(q_k)$$

2. Puisque y est un vecteur *one-hot*, $y_c = 1$ et $y_k = 0$ pour tout $k \neq c$. La somme se réduit à un seul terme non nul :

$$H(y, \hat{y}) = -\ln(q_c)$$

La fonction de perte de la classification multi-classe dépend uniquement du logarithme de la probabilité que le modèle assigne à la bonne classe c. Augmenter cette probabilité minimise directement la Cross-Entropy.

Exercice 7 : Convexité de l'Entropie Croisée

★★★★⋆

On étudie la cross-entropy en tant que fonction de la distribution prédite q. Soit une distribution fixe P sur un ensemble fini d'états $i \in \{1, \ldots, N\}$ avec des probabilités $p_i > 0$. Soit la fonction $f : \Delta \to \mathbb{R}$ définie sur le simplexe de probabilité $\Delta = \{q = (q_1, \ldots, q_N) \in \mathbb{R}^N : \sum q_i = 1, q_i > 0\}$ par :

$$f(q) = -\sum_{i=1}^{N} p_i \ln(q_i)$$

1. Calculer le gradient et la matrice Hessienne de la fonction f par rapport au vecteur q. **2.** En déduire que la fonction de perte Cross-Entropy est strictement convexe par rapport aux probabilités prédites.

Correction :

1. Calculons les dérivées partielles premières :

$$\frac{\partial f}{\partial q_k} = -\frac{p_k}{q_k}$$

Le gradient est donc $\nabla f(q) = \left(-\frac{p_1}{q_1}, \ldots, -\frac{p_N}{q_N}\right)^T$.

Calculons les dérivées partielles secondes : Si $j \neq k$, $\frac{\partial^2 f}{\partial q_k \partial q_j} = 0$. Si $j = k$, $\frac{\partial^2 f}{\partial q_k^2} = \frac{\partial}{\partial q_k}\left(-\frac{p_k}{q_k}\right) = \frac{p_k}{q_k^2}$. La matrice Hessienne $H_f(q)$ est donc une matrice diagonale :

$$H_f(q) = \text{Diag}\left(\frac{p_1}{q_1^2}, \ldots, \frac{p_N}{q_N^2}\right)$$

2. Pour démontrer la stricte convexité, il suffit de montrer que la matrice Hessienne est définie positive. Soit un vecteur non nul $v \in \mathbb{R}^N$. Considérons la forme quadratique associée :

$$v^T H_f(q) v = \sum_{i=1}^{N} \frac{p_i}{q_i^2} v_i^2$$

Puisque par hypothèse $p_i > 0$ et $q_i > 0$, chaque terme de la somme est strictement positif si $v_i \neq 0$. Puisque v est non nul, la somme entière est strictement positive. La Hessienne est donc définie positive sur tout le simplexe Δ. Par conséquent, la fonction de perte Cross-Entropy est strictement convexe. Cela garantit qu'il existe un unique minimum global, qui correspond à $q = p$.

Exercice 8 : Théorème de convergence dominée pour la LFGN

★★★★⋆

Lors de la preuve de la convergence de la perte empirique, nous avons supposé $\mathbb{E}_{X\sim P}[|\ln q_\theta(X)|] < \infty$. Supposons un espace compact $\mathcal{X}$, et que les densités $p(x)$ et $q_\theta(x)$ soient continues, strictement positives sur $\mathcal{X}$.

1. Montrer que l'hypothèse d'intégrabilité est automatiquement satisfaite sous ces conditions.
2. Comment cela assure-t-il la validité de la Loi Forte des Grands Nombres pour l'entropie croisée ?

Correction :

1. Puisque $q_\theta(x)$ est une fonction continue et strictement positive sur un espace compact $\mathcal{X}$, d'après le théorème des bornes atteintes de Weierstrass, elle atteint son minimum $m > 0$ et son maximum $M < \infty$ sur $\mathcal{X}$. On a donc pour tout $x \in \mathcal{X}$, $0 < m \leq q_\theta(x) \leq M$. Par croissance stricte et continuité du logarithme :

$$\ln(m) \leq \ln q_\theta(x) \leq \ln(M)$$

Ainsi, la fonction $|\ln q_\theta(x)|$ est bornée par une constante $K = \max(|\ln(m)|, |\ln(M)|)$. Par conséquent, l'espérance est majorée :

$$\mathbb{E}_{X\sim P}[|\ln q_\theta(X)|] = \int_{\mathcal{X}} p(x)|\ln q_\theta(x)| d\lambda(x) \leq K \int_{\mathcal{X}} p(x) d\lambda(x) = K < \infty$$

L'hypothèse d'intégrabilité absolue est donc satisfaite.

2. La condition L^1 (espérance de la valeur absolue finie) est la condition nécessaire et suffisante du théorème fondamental de Kolmogorov (Loi Forte des Grands Nombres) pour une suite de variables i.i.d. $Y_i = -\ln q_\theta(X_i)$. Elle assure que la moyenne empirique converge presque sûrement vers son espérance théorique, validant ainsi la minimisation asymptotique.

Exercice 9 : Entropie d'une distribution Gaussienne Multivariée

★★★★★

Soit un vecteur aléatoire $X \in \mathbb{R}^d$ suivant une distribution normale multivariée $\mathcal{N}(\mu, \Sigma)$, avec Σ une matrice de covariance symétrique définie positive. La densité de probabilité est :

$$p(x) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right)$$

1. Démontrer que l'entropie différentielle de cette source est $H(X) = \frac{1}{2}\ln\left((2\pi e)^d \det(\Sigma)\right)$. **2.** Si un modèle approximait cette source avec une gaussienne d'une autre covariance, qu'adviendrait-il de l'entropie croisée ?

Correction :

1. L'entropie différentielle est $H(X) = \mathbb{E}_{X\sim P}[-\ln p(X)]$. Calculons $-\ln p(X)$:

$$-\ln p(X) = \ln\left(\sqrt{(2\pi)^d \det(\Sigma)}\right) + \frac{1}{2}(X-\mu)^T \Sigma^{-1}(X-\mu)$$

L'espérance est linéaire, donc :

$$H(X) = \frac{1}{2}\ln\left((2\pi)^d \det(\Sigma)\right) + \frac{1}{2}\mathbb{E}\left[(X-\mu)^T \Sigma^{-1}(X-\mu)\right]$$

Le second terme est l'espérance d'une forme quadratique. En utilisant la trace, on a :

$$\mathbb{E}\left[(X-\mu)^T \Sigma^{-1}(X-\mu)\right] = \mathbb{E}\left[\mathrm{Tr}\left((X-\mu)^T \Sigma^{-1}(X-\mu)\right)\right]$$

Par invariance cyclique de la trace :

$$\mathbb{E}\left[\mathrm{Tr}\left(\Sigma^{-1}(X-\mu)(X-\mu)^T\right)\right] = \mathrm{Tr}\left(\Sigma^{-1}\mathbb{E}[(X-\mu)(X-\mu)^T]\right)$$

Or, par définition de la matrice de covariance, $\mathbb{E}[(X-\mu)(X-\mu)^T] = \Sigma$. Donc :

$$\mathrm{Tr}(\Sigma^{-1}\Sigma) = \mathrm{Tr}(I_d) = d$$

Ainsi, l'entropie est :

$$H(X) = \frac{1}{2}\ln\left((2\pi)^d \det(\Sigma)\right) + \frac{d}{2}$$

Puisque $\frac{d}{2} = \frac{1}{2}\ln(e^d)$, nous pouvons regrouper les logarithmes :

$$H(X) = \frac{1}{2}\ln\left((2\pi e)^d \det(\Sigma)\right)$$

2. Si le modèle approxime la source avec une distribution normale de même moyenne mais de covariance Σ', la divergence KL sera non nulle. L'entropie croisée sera strictement supérieure à $H(X)$ (la perte sera sous-optimale), mesurant le coût d'encoder la véritable distribution avec des paramètres de covariance incorrects.

Exercice 10 : Inégalité de Gibbs et Borne de l'Entropie Croisée

★★★★★

Soit deux distributions de probabilité discrètes $P = (p_1, \ldots, p_N)$ et $Q = (q_1, \ldots, q_N)$ avec $p_i, q_i > 0$. L'inégalité de Gibbs stipule que $-\sum_{i=1}^N p_i \ln p_i \leq -\sum_{i=1}^N p_i \ln q_i$.

1. Démontrer cette inégalité de manière élémentaire, en utilisant l'inégalité fondamentale $\ln x \leq x - 1$ pour tout $x > 0$. **2.** Déterminer le cas précis où l'égalité est vérifiée.

Correction :

1. Soit l'expression de la différence entre l'entropie et l'entropie croisée (qui n'est autre que l'opposé de la divergence KL) :

$$-\sum_{i=1}^{N} p_i \ln q_i - \left(-\sum_{i=1}^{N} p_i \ln p_i\right) = -\sum_{i=1}^{N} p_i (\ln q_i - \ln p_i) = -\sum_{i=1}^{N} p_i \ln\left(\frac{q_i}{p_i}\right)$$

Nous utilisons l'inégalité fondamentale pour le logarithme népérien : $\ln(x) \leq x - 1$ (c'est une propriété de la concavité du logarithme : la tangente en $x = 1$ est $y = x - 1$). Ainsi, $\ln\left(\frac{q_i}{p_i}\right) \leq \frac{q_i}{p_i} - 1$. En introduisant cela dans la somme :

$$-\sum_{i=1}^{N} p_i \ln\left(\frac{q_i}{p_i}\right) \geq -\sum_{i=1}^{N} p_i \left(\frac{q_i}{p_i} - 1\right)$$

Développons le terme de droite :

$$-\sum_{i=1}^{N} p_i \left(\frac{q_i}{p_i} - 1\right) = -\sum_{i=1}^{N} (q_i - p_i) = -\left(\sum_{i=1}^{N} q_i - \sum_{i=1}^{N} p_i\right)$$

Puisque P et Q sont des distributions de probabilité valides, la somme de leurs composantes vaut 1 : $\sum p_i = 1$ et $\sum q_i = 1$. Par conséquent, la différence vaut :

$$-(1 - 1) = 0$$

Nous avons donc démontré que la différence entre l'entropie croisée et l'entropie est toujours positive ou nulle, soit :

$$H(P, Q) \geq H(P)$$

2. L'égalité $\ln x = x - 1$ n'est vérifiée que si et seulement si $x = 1$. Dans notre cas, cela signifie que pour chaque $i \in \{1, \ldots, N\}$, nous devons avoir $\frac{q_i}{p_i} = 1$, ce qui équivaut à $q_i = p_i$. L'égalité est donc vérifiée si et seulement si les deux distributions sont strictement identiques p.p.

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

TP 1 : Calcul empirique de l'Entropie de Shannon

Ce premier TP consiste à coder sans framework externe le calcul de l'entropie de Shannon sur une distribution discrète.

Objectif : Écrire une fonction Python robuste qui, étant donné un dictionnaire de probabilités ou une liste de comptages de fréquences, retourne l'entropie de la source en bits.

```python
import math

def calculate_shannon_entropy(frequencies):
    total = sum(frequencies.values())

    # Normalisation pour obtenir des probabilites
    probabilities = {k: v / total for k, v in frequencies.items()}

    entropy = 0.0
    for p in probabilities.values():
        if p > 0:
            entropy -= p * math.log2(p)

    return entropy

# Validation mathematique avec une source uniforme
uniform_dist = {'A': 25, 'B': 25, 'C': 25, 'D': 25}
ent_uniform = calculate_shannon_entropy(uniform_dist)

# Une source uniforme sur 4 symboles donne log2(4) = 2 bits
assert math.isclose(ent_uniform, 2.0), "Echec de l'assertion pour la distribution uniforme."

print("Entropie uniforme: ", ent_uniform, "bits")

# Source non uniforme
biased_dist = {'A': 50, 'B': 25, 'C': 12.5, 'D': 12.5}
ent_biased = calculate_shannon_entropy(biased_dist)
print("Entropie biaisee: ", ent_biased, "bits")
assert ent_biased < ent_uniform, "L'entropie d'une distribution biaisee doit etre inferieure."
```

TP 2 : Implémentation de la Cross-Entropy et Divergence KL

Le but de ce TP est de calculer la fonction de perte "Cross-Entropy" entre une distribution réelle et une distribution prédite, et d'en vérifier la relation fondamentale avec l'entropie de Shannon et la divergence KL.

Objectif : Écrire la fonction de cross-entropy et la fonction KL.

```python
import math

def cross_entropy(p_dist, q_dist):
```

```
    # Verifier que les cles correspondent
    assert set(p_dist.keys()) == set(q_dist.keys()), "Les espaces d'
        etats different."

    ce = 0.0
    for state in p_dist:
        p = p_dist[state]
        q = q_dist[state]
        if p > 0:
            if q == 0:
                return float('inf')
            ce -= p * math.log2(q)
    return ce

def kl_divergence(p_dist, q_dist):
    kl = 0.0
    for state in p_dist:
        p = p_dist[state]
        q = q_dist[state]
        if p > 0:
            if q == 0:
                return float('inf')
            kl += p * math.log2(p / q)
    return kl

# Distribution reelle (P)
P = {'A': 0.5, 'B': 0.25, 'C': 0.25}

# Modele parfait
Q_perfect = {'A': 0.5, 'B': 0.25, 'C': 0.25}

# Modele imparfait (uniforme)
Q_uniform = {'A': 1/3, 'B': 1/3, 'C': 1/3}

ce_perfect = cross_entropy(P, Q_perfect)
ce_uniform = cross_entropy(P, Q_uniform)

# Validation des theoremes
from math import isclose
H_P = sum(-p * math.log2(p) for p in P.values() if p > 0)

assert isclose(ce_perfect, H_P), "La cross-entropy d'un modele parfait
    vaut l'entropie."
assert ce_uniform > H_P, "La cross-entropy d'un modele imparfait doit
    depasser l'entropie."

kl = kl_divergence(P, Q_uniform)
assert isclose(ce_uniform, H_P + kl), "H(P, Q) = H(P) + D_KL(P || Q) non
     verifie."
print("Theoreme mathematique valide : H(P, Q) = H(P) + KL")
```

TP 3 : Simulation de la convergence de la perte empirique (LFGN)

Ce TP valide numériquement le Théorème de convergence de la perte empirique vers la cross-entropy théorique.

Objectif : Générer un échantillon I.I.D. issu d'une distribution P, puis calculer la "Negative Log-Likelihood" moyenne (perte empirique) selon un modèle Q, et tracer la convergence.

```python
import random
import math

random.seed(42)

# Vraies probabilites P
P_probs = [0.1, 0.2, 0.7]
states = [0, 1, 2]

# Modele Q arbitraire
Q_probs = {0: 0.3, 1: 0.3, 2: 0.4}

# Valeur theorique de la Cross-Entropy
H_PQ_theo = - (0.1 * math.log2(0.3) + 0.2 * math.log2(0.3) + 0.7 * math.log2(0.4))

# Generation d'un echantillon selon P
N = 10000
samples = random.choices(states, weights=P_probs, k=N)

# Suivi de la perte moyenne au fil des echantillons
empirical_loss = []
current_sum = 0.0

for i, x_i in enumerate(samples):
    q_x = Q_probs[x_i]
    loss_i = - math.log2(q_x)
    current_sum += loss_i

    if (i + 1) % 100 == 0:
        empirical_loss.append(current_sum / (i + 1))

# Validation mathematique de la convergence (Loi Forte des Grands Nombres)
final_empirical = current_sum / N
print(f"Cross-Entropy Theorique : {H_PQ_theo:.4f}")
print(f"Perte Empirique sur {N} ech. : {final_empirical:.4f}")

assert math.isclose(H_PQ_theo, final_empirical, abs_tol=0.05), "La perte empirique ne converge pas vers la valeur theorique."
```

TP 4 : Perplexité et Softmax sur des logits bruts

Les modèles de langage modernes produisent des scores bruts (logits) qui sont convertis en probabilités via la fonction Softmax. Ce TP implémente le chemin complet Logits → Probabilités → Perte Cross-Entropy → Perplexité.

Objectif : Écrire des fonctions numériquement stables.

```python
import math

def softmax(logits):
    # Stabilite numerique : on soustrait le max
```

```
    m = max(logits)
    exp_vals = [math.exp(x - m) for x in logits]
    sum_exp = sum(exp_vals)
    return [v / sum_exp for v in exp_vals]

def cross_entropy_loss_from_logits(logits, target_index):
    # Le modele calcule la perte pour UN echantillon (one-hot)
    probs = softmax(logits)
    # L'entropie croisee se reduit au log de la proba de la vraie classe
    return -math.log(probs[target_index])

# Simulation d'un batch
# Vocabulaire de 4 mots
logits_batch = [
    [2.0, -1.0, 0.5, 0.1],
    [0.1, 3.0, 0.1, 0.1],
    [-2.0, -2.0, 5.0, 0.0]
]
targets = [0, 1, 2] # Les indices des bons mots

total_loss = 0.0
for i in range(len(targets)):
    loss = cross_entropy_loss_from_logits(logits_batch[i], targets[i])
    total_loss += loss

avg_loss = total_loss / len(targets)
perplexity = math.exp(avg_loss)

print(f"Perte moyenne (nats): {avg_loss:.4f}")
print(f"Perplexite du modele: {perplexity:.4f}")

# Validation mathematique : un modele parfait (loss -> 0) a une
    perplexite -> 1
assert perplexity >= 1.0, "La perplexite ne peut pas etre inferieure a
    1."
```

TP 5 : Descente de gradient manuelle sur la Cross-Entropy

Dans ce dernier TP d'implémentation bas niveau, nous allons optimiser les poids d'une régression logistique binaire pour minimiser la fonction Cross-Entropy.

Objectif : Prouver que la fonction est convexe et que la descente de gradient trouve le minimum.

```
import math

# Echantillon : 2 points en 1D (x) avec labels binaires (y)
data = [(1.0, 0), (2.0, 0), (8.0, 1), (9.0, 1)]

# Le modele : probabilite P(Y=1 | x) = sigmoid(w * x + b)
def sigmoid(z):
    if z < -20: return 0.0
    if z > 20: return 1.0
    return 1.0 / (1.0 + math.exp(-z))

# Poids initiaux
```

```
w = 0.0
b = 0.0
lr = 0.5
epochs = 500

for epoch in range(epochs):
    dw = 0.0
    db = 0.0
    loss = 0.0

    for x, y_true in data:
        y_pred = sigmoid(w * x + b)

        # Securite pour le logarithme
        y_pred = max(1e-10, min(1 - 1e-10, y_pred))

        # Accumulation de la Cross-Entropy
        loss += - (y_true * math.log(y_pred) + (1 - y_true) * math.log(1
            - y_pred))

        # Gradient de la CE : propriete remarquable dL/dz = y_pred -
           y_true
        diff = y_pred - y_true
        dw += diff * x
        db += diff * 1.0

    # Moyenne
    loss /= len(data)
    dw /= len(data)
    db /= len(data)

    # Mise a jour
    w -= lr * dw
    b -= lr * db

print(f"Fin d'entrainement. Poids : w={w:.4f}, b={b:.4f}")
print(f"Perte finale : {loss:.4f}")

# Validation mathematique du separateur (frontiere de decision -b/w)
boundary = -b / w
print(f"Frontiere de decision : {boundary:.2f}")

# La frontiere doit parfaitement separer les classes entre x=2 et x=8
assert 2.0 < boundary < 8.0, "La descente de gradient n'a pas trouve l'
   optimum global."
```